KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 8 Quiz

Veritabanı: SQLAlchemy ve Kalıcılık

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

`EtkinlikDB` sınıfındaki `id: Mapped[int] = mapped_column(primary_key=True)` satırı tam olarak neyi tanımlar; bu, Hf.6'nın Pydantic `BaseModel`'indeki tip ipucu kullanımıyla nasıl kıyaslanabilir?

(a) Yalnızca IDE'ye ipucu vermek amaçlıdır; SQLAlchemy çalışma zamanında bu tip ipucunu tamamen görmezden gelir, sütun tipini ayrı bir yerde elle belirtmeniz gerekir.

(b) Bu satır yalnızca Python nesnesinin bir özniteliğini tanımlar; veritabanı tablosuyla hiçbir ilişkisi yoktur, tablo şeması `create_all` çağrısında tamamen ayrı biçimde elle yazılır.

(c) Tip ipucu (`Mapped[int]`) ile `mapped_column(...)` birlikte, `etkinlikler` tablosunda bir tamsayı birincil anahtar sütunu tanımlar — Pydantic'in tip ipucundan doğrulama üretmesine benzer biçimde, burada tip ipucundan bir sütun tanımı üretilir.

(d) `Mapped[int]` yalnızca bu alanın asla `None` olamayacağını belirtir; birincil anahtar olma özelliği yalnızca `mapped_column(primary_key=True)` tarafından, `Mapped` tipinden bağımsız sağlanır.

Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

Bir rota fonksiyonu, `depo.etkinlik_getir(...)` çağrısından dönen bir `EtkinlikDB` nesnesini FastAPI'ye doğrudan JSON olarak döndürmek yerine, önce `Etkinlik.model_validate(etkinlik_db, from_attributes=True)` ile bir Pydantic nesnesine neden çevirir?

(a) `EtkinlikDB` bir veritabanı satırının Python temsilidir; `Etkinlik` ise ondan tamamen bağımsız, dışa dönük bir JSON sözleşmesidir.

(b) `EtkinlikDB` sınıfı JSON serileştirmesini zaten otomatik yapar; `model_validate` çağrısı yalnızca performans amaçlı, isteğe bağlı bir optimizasyondur.

(c) FastAPI, `EtkinlikDB` gibi SQLAlchemy sınıflarını `response_model` olarak hiçbir koşulda kabul etmez; bu yalnızca bir kütüphane kısıtlamasıdır, katmanların bilinçli olarak ayrılmasıyla hiçbir ilgisi yoktur.

(d) `model_validate` çağrısı yalnızca alan adlarını camelCase'e çevirir; `EtkinlikDB`'nin kendisi zaten JSON'a serileştirilebilir durumdadır, dönüşüm gereksizdir.

Soru 3 (8 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki kesitler `app/orm_modeller.py` ve `app/veritabani.py` dosyalarından alınmıştır.

```python
class KoltukDB(Base):
    """Bir `koltuklar` satırının Python temsili — HER koltuk bir etkinliğe AİTTİR."""

    __tablename__ = "koltuklar"

    id: Mapped[int] = mapped_column(primary_key=True)
    etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"))
    sira: Mapped[int]
    koltuk_no: Mapped[int]
    durum: Mapped[str] = mapped_column(default="bos")

    etkinlik: Mapped["EtkinlikDB"] = relationship(back_populates="koltuklar")
    biletler: Mapped[list["BiletDB"]] = relationship(
        back_populates="koltuk", cascade="all, delete-orphan"
    )
```

```python
def get_db() -> Generator[Session, None, None]:
    """Her istek için YENİ bir Session açar, istek bitince KAPATIR.

    `yield`'den ÖNCEKİ blok isteği İŞLEYEN koddan ÖNCE, `finally` bloğu
    isteği İŞLEYEN kod bittikten SONRA (hata OLSA bile) çalışır.
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

(a) `KoltukDB`'de `durum: Mapped[str] = mapped_column(default="bos")` satırı var, ama `app/modeller.py`'deki `KoltukOlustur` Pydantic modelinde `durum` alanı hiç yoktur. Bu tutarsız bir tasarım mıdır, yoksa bilinçli bir seçim midir; neden? (b) `get_db()` içindeki isteği işleyen rota kodu bir hata fırlatırsa (ör. bir `HTTPException`), `db.close()` yine de çalışır mı? `finally` bloğunun burada kullanılmasının nedeni nedir?

Soru 4 (10 puan)

4 • Değerlendirme Hedef 1

`EtkinlikDB` (ORM modeli) ile `Etkinlik` (Pydantic modeli) neden aynı sınıf değildir; `model_validate(from_attributes=True)` ne işe yarar? Cevabınızda iki modelin taşıdığı farklı sorumlulukları ayırt ediniz.

Soru 5 (6 puan)

4 • Değerlendirme Hedef 2

`etkinlik.koltuklar` ifadesine eriştiğinizde, SQLAlchemy'nin arka planda yaptığı şey nedir; bu, elle yazacağınız hangi SQL sorgusuna karşılık gelir?

(a) `koltuklar` alanı, `EtkinlikDB` nesnesi oluşturulduğu anda, ilgili tüm koltuk verisinin belleğe önceden kopyalanmış statik bir Python listesidir; erişim hiçbir SQL sorgusu tetiklemez.

(b) Bu erişim yalnızca `relationship` tanımının doğru yazıldığını denetleyen bir tip kontrolüdür; gerçek veri yalnızca `selectinload` açıkça çağrıldığında SQL'den okunur.

(c) `koltuklar`'a erişim, `etkinlikler` tablosundaki tüm satırları tarayan bir `SELECT * FROM etkinlikler` sorgusu tetikler; N+1 sorununun asıl kaynağı bu erişimdir.

(d) SQLAlchemy arka planda `WHERE etkinlik_id = ?` biçiminde bir `SELECT` sorgusu çalıştırır ve sonucu bir Python listesi olarak döndürür; siz elle bir JOIN/WHERE yazmak zorunda kalmazsınız.

Soru 6 (6 puan)

4 • Değerlendirme Hedef 2

`DELETE /etkinlikler/{id}` isteği, o etkinliğe bağlı koltuklardan birinin aktif (`durum='olusturuldu'`) bir bileti varsa, hangi durum koduyla, neden sonuçlanır?

(a) `204 No Content` ile sonuçlanır; etkinlik ve bağlı tüm koltuk/bilet kayıtları, satılmış olup olmadığına bakılmaksızın otomatik silinir.

(b) `409 Conflict` ile sonuçlanır; istek format olarak geçerlidir ama etkinliği silmek mevcut durumla (satılmış biletler var) çelişir — silme reddedilir, önce biletlerin iptal edilmesi gerekir.

(c) `404 Not Found` ile sonuçlanır; sunucu, satılmış biletleri olan bir etkinliği veritabanında hiç bulunamamış gibi ele alır.

(d) `422 Unprocessable Entity` ile sonuçlanır; sunucu isteği bir doğrulama (*validation*) hatası olarak ele alır, çünkü `etkinlik_id` biçimi geçersiz sayılır.

Soru 7 (8 puan)

4 • Değerlendirme Hedef 2

Aşağıdaki kesitler `app/depo.py` ve `app/orm_modeller.py` dosyalarından alınmıştır.

```
def etkinlik_sil(self, etkinlik_id: int) -> str:
    """Etkinliği SİLER (cascade) ya da neden silinemediğini bildirir.

    Dönüş: `"silindi"` | `"bulunamadi"` | `"aktif_bilet_var"`. Bağlı
    koltuklardan HERHANGİ BİRİNİN aktif (`durum='olusturuldu'`) bir
    bileti varsa silme REDDEDİLİR; yoksa `relationship`'teki
    `cascade="all, delete-orphan"` sayesinde bağlı TÜM koltuklar (ve
    varsa iptal edilmiş biletleri) OTOMATİK olarak kademeli silinir.
    """
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return "bulunamadi"

    aktif_bilet_var = any(
        bilet.durum == "olusturuldu"
```

Soru 7 (devamı)

```
        for koltuk in etkinlik_db.koltuklar
        for bilet in koltuk.biletler
    )
    if aktif_bilet_var:
        return "aktif_bilet_var"

    self.db.delete(etkinlik_db)
    self.db.commit()
    return "silindi"
```

```
class EtkinlikDB(Base):
    """Bir `etkinlikler` satırının Python temsili."""

    __tablename__ = "etkinlikler"

    id: Mapped[int] = mapped_column(primary_key=True)
    baslik: Mapped[str]
    tarih: Mapped[str]
    mekan: Mapped[str]
    kapasite: Mapped[int]
    bilet_fiyati: Mapped[float]

    # Bir etkinlik SİLİNDİĞİNDE bağlı TÜM koltuklar (ve onların cascade'ine
    # bağlı biletleri) OTOMATİK silinir — Ünite 2'nin cascade silme kuralı.
    koltuklar: Mapped[list["KoltukDB"]] = relationship(
        back_populates="etkinlik", cascade="all, delete-orphan"
    )
```

(a) `aktif_bilet_var` Değişkeni nasıl hesaplanır — hangi iki iç içe döngüyü kullanır, hangi bilet `durum` değerini kontrol eder? (b) Aktif bilet yoksa `self.db.delete(etkinlik_db)` çağrıldığında, bağlı koltuklar (ve varsa iptal edilmiş biletleri) neden ayrıca tek tek silinmesine gerek kalmadan otomatik silinir?

Soru 8 (10 puan)

4 • Değerlendirme Hedef 2

`GET /etkinlikler` isteğinde her etkinliğin koltuk sayısını ayrı ayrı bir sorguyla çekerseniz (tembel yükleme, N etkinlik için N ek sorgu) nasıl bir sorun oluşur; bu sorun nasıl önlenir?

Soru 9 (6 puan)

4 • Değerlendirme Hedef 3

`GET /etkinlikler?sayfa=2&boyut=10` isteği, `Depo.etkinlik_listele` içinde hangi `limit`/`offset` değerleriyle sonuçlanır?

(a) `limit(10)`, `offset(10)` — `offset`, `(sayfa - 1) * boyut` formülüyle hesaplanır; ilk 10 kayıt (1. sayfa) atlanır, 11–20. kayıtlar döner.

(b) `limit(10)`, `offset(2)` — `offset` doğrudan `sayfa` parametresinin kendisidir.

(c) `limit(2)`, `offset(10)` — `limit` değeri `sayfa` parametresinden, `offset` değeri `boyut` parametresinden alınır.

(d) `limit(20)`, `offset(0)` — `sayfa`/`boyut` sorgu parametreleri yalnızca varsayılan değerler değiştirilmediği sürece etkilidir, bu istekte göz ardı edilir.

Soru 10 (8 puan)

4 • Değerlendirme Hedef 3

Aşağıdaki kesitler `app/depo.py` ve `sorgular.sql` dosyalarından alınmıştır.

```python
def etkinlik_ekle(self, veri: EtkinlikOlustur) -> Etkinlik:
    etkinlik_db = EtkinlikDB(**veri.model_dump())
    self.db.add(etkinlik_db)
    self.db.commit()
    self.db.refresh(etkinlik_db)
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)

def etkinlik_getir(self, etkinlik_id: int) -> Etkinlik | None:
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return None
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)

def etkinlik_listele(
    self, sayfa: int = 1, boyut: int = 20, mekan: str | None = None
) -> list[Etkinlik]:
    stmt = select(EtkinlikDB).order_by(EtkinlikDB.id)
    if mekan is not None:
        stmt = stmt.where(EtkinlikDB.mekan == mekan)
    stmt = stmt.limit(boyut).offset((sayfa - 1) * boyut)
    etkinlikler_db = self.db.execute(stmt).scalars().all()
    return [
        Etkinlik.model_validate(e, from_attributes=True) for e in etkinlikler_db
    ]

def etkinlik_guncelle(
    self, etkinlik_id: int, veri: EtkinlikOlustur
) -> Etkinlik | None:
    etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
    if etkinlik_db is None:
        return None
    for alan, deger in veri.model_dump().items():
        setattr(etkinlik_db, alan, deger)
    self.db.commit()
    self.db.refresh(etkinlik_db)
    return Etkinlik.model_validate(etkinlik_db, from_attributes=True)
```

```sql
-- GET /etkinlikler?sayfa=2&boyut=10 karşılığı
```

Soru 10 (devamı)

```sql
SELECT etkinlikler.id, etkinlikler.baslik, etkinlikler.tarih,
     etkinlikler.mekan, etkinlikler.kapasite, etkinlikler.bilet_fiyati
FROM etkinlikler
ORDER BY etkinlikler.id
LIMIT 10 OFFSET 10;
```

(a) `etkinlik_listele(sayfa=2, boyut=10, mekan=None)` çağrısında `stmt.where(EtkinlikDB.mekan == mekan)` satırı çalışır mı? Neden? (b) `sorgular.sql`'deki `sql_sayfalama` kesiti ile `etkinlik_listele`'nin ürettiği SQL, işlevsel olarak aynı sonucu üretir mi; aralarındaki tek fark nedir?

Soru 11 (10 puan)

4 • Değerlendirme Hedef 3

`echo=True` Açıkken ORM'in ürettiği SQL, YG II'de elle yazdığınız SQL'den neden farklı görünebilir (ör. takma adlar, sütun sırası) — bu bir sorun mudur? `text()` ile ham SQL çalıştırmak ne zaman gerekir?

Soru 12 (6 puan)

4 • Değerlendirme Hedef 4

`POST /biletler` isteğinde istenen koltuk zaten dolu ise (`koltuk_db.durum == "dolu"`), `Depo.bilet_ekle` neden `400 Bad Request` değil `409 Conflict` fırlatır?

(a) `400` yalnızca kimlik doğrulama hatalarında kullanılır; bu istek kimlik doğrulama gerektirmediği için `400` kullanılamaz, bu yüzden `409` seçilmiştir.

(b) `409` ve `400` durum kodları FastAPI'de birbirinin yerine geçerlidir; hangisinin kullanıldığı yalnızca bir isimlendirme tercihidir, kütüphane içinde herhangi bir davranış farkı oluşturmaz, istemci tarafını da hiçbir şekilde etkilemez.

(c) İstek gövdesi format olarak geçerlidir; sorun mevcut durumla (koltuk satılmış) çelişmesidir — bu `409 Conflict`'in tanımıdır.

(d) `409` yalnızca veritabanı bağlantısı kesildiğinde kullanılan bir durum kodudur; bu senaryoda kullanılması bir hatadır, doğrusu `503 Service Unavailable` olmalıdır.

Soru 13 (6 puan)

4 • Değerlendirme Hedef 4

`bilet_ekle` metodundaki "koltuğu kontrol et, dolu yap, bileti yaz" adımları neden ayrı ayrı `commit` edilmez, tek bir transaction sınırında tutulur?

Soru 13 (devamı)

(a) Performans nedeniyle; ayrı ayrı commit etmek SQLite dosyasını gereksiz yere büyütür, tek commit disk alanından tasarruf sağlar.

(b) Adımlar ayrı ayrı commit edilseydi, iki commit arasındaki boşlukta başka bir isteğin araya girip aynı koltuğu alması mümkün olurdu; tek transaction bu boşluğu tamamen kapatır, "ya hep ya hiç" garantisi verir.

(c) SQLAlchemy'nin `Session` API'si teknik olarak ardışık birden fazla `commit()` çağrısına izin vermez; bu yüzden zorunlu olarak tek commit kullanılır.

(d) Tek commit kullanılması yalnızca test ortamında (`:memory:` veritabanında) gerekliliktir; üretim ortamındaki gerçek `bilet.db` dosyasında adımların ayrı ayrı commit edilmesinin hiçbir sakıncası yoktur.

Soru 14 (8 puan)

4 • Değerlendirme Hedef 4

Aşağıdaki kesitler `app/depo.py` ve `tests/test_transaction.py` dosyalarından alınmıştır.

```python
def bilet_ekle(self, veri: BiletOlustur) -> Bilet:
    """Koltuk kontrolü + koltuğu doldurma + bilet ekleme TEK transaction'da.

    Bu Session'ın bağlı olduğu `engine`, her transaction'ı SQLite'ın
    `BEGIN IMMEDIATE`'iyle başlatır (`app/veritabani.py`) — aynı koltuğa
    eşzamanlı gelen ikinci istek, birincisi commit edene kadar burada
    BEKLER ve koltuğun ARTIK dolu olduğunu görüp `409` alır. Uygulama
    kontrolü (`durum == 'dolu'`) herhangi bir nedenle atlanırsa, kısmi
    UNIQUE indeks (`orm_aktif_koltuk_indeksi`) bir `IntegrityError`
    fırlatır — bu da `409`'a çevrilir; "tek kalem kuralı"nın veritabanı
    seviyesinde TEKRARLANMASI.
    """
    koltuk_db = self.db.get(KoltukDB, veri.koltuk_id)
    if koltuk_db is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    if koltuk_db.durum == "dolu":
        raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")

    koltuk_db.durum = "dolu"
    bilet_db = BiletDB(
        koltuk_id=veri.koltuk_id,
        kullanici_adi=veri.kullanici_adi,
        satin_alma_zamani=datetime.now().isoformat(),
        durum="olusturuldu",
    )
    self.db.add(bilet_db)

    try:
        self.db.commit()
    except IntegrityError:
        self.db.rollback()
        raise HTTPException(
            status_code=409, detail="Bu koltuk zaten satılmış"
        ) from None
    self.db.refresh(bilet_db)
    return Bilet.model_validate(bilet_db, from_attributes=True)
```

```python
def test_ardisik_rezervasyon_409(client):
```

Soru 14 (devamı)

```python
    """Aynı koltuk için ARDIŞIK iki POST /biletler isteği — ikincisi 409 döner."""
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)

    ilk = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    )
    assert ilk.status_code == 201

    ikinci = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Mehmet Demir"}
    )
    assert ikinci.status_code == 409
    assert "satılmış" in ikinci.json()["detail"]

    # Sunucuda TEK bir bilet vardır — ikinci isteğin YAZDIĞI hiçbir kayıt YOK.
    biletler = client.get("/biletler").json()
    assert len(biletler) == 1
```

(a) `except IntegrityError:` bloğunda neden önce `self.db.rollback()` çağrılır, sonra `HTTPException(409, ...)` fırlatılır — `rollback()` atlansaydı ne olurdu? (b) `test_ardisik_rezervasyon_409` testindeki son satır (`len(biletler) == 1`) tam olarak neyi kanıtlıyor; bu satır olmasaydı test hangi hatayı kaçırabilirdi?

Soru 15 (10 puan)

4 • Değerlendirme **Hedef 4**

Aynı koltuğa aynı anda gönderilen iki `POST /biletler` isteğinde, `BEGIN IMMEDIATE` transaction başlatma biçimi yarış durumunu (*race condition*) tam olarak nasıl önler? İki gişe memuru benzetmesini kullanarak açıklayınız; kısmi `UNIQUE` indeksin bu korumaya nasıl bir ikinci savunma katmanı eklediğini de belirtiniz.